

KREACIJSKI DIZAJN PATERNI

Uvod

Kreacijski dizajn paterni koriste se za organizovanje kreiranje objekata u sistemu. Njihova svrha je da se smanji zavisnost između klasa i da se izbjegne direktno kreiranje objekata na više različitih mjesta u kodu.

U sistemu pametnog parkinga analizirani su različiti kreacijski paterni, a na novi klasni dijagram su dodana dva paterna koja se najbolje uklapaju u postojeći sistem: Factory Method i Builder.

1. Singleton

Singleton patern se koristi kada u sistemu treba postojati samo jedna instanca neke klase. Najčešće se koristi za logovanje, konfiguraciju sistema ili upravljanje zajedničkim resursima.

U našem sistemu pametnog parkinga Singleton bi se mogao koristiti za klasu koja vodi evidenciju sistemskih aktivnosti, npr. logovanje grešaka prilikom rezervacije, plaćanja ili slanja email obavijesti.

Korištenje:

Singleton bi omogućio da cijeli sistem koristi isti objekat za zapisivanje važnih događaja i grešaka. Ovaj patern nije dodat na klasni dijagram jer nije direktno vezan za glavne funkcionalnosti rezervacije i obavijesti koje su obrađene u ovom zadatku.

2. Factory Method

Factory Method patern koristi se kada kreiranje objekta zavisi od određenog tipa ili uslova. Umjesto da se objekti kreiraju direktno u controlleru ili drugim klasama, kreiranje se prebacuje u posebnu factory klasu.

U našem sistemu ovaj patern je odabran i dodat na klasni dijagram kod email obavijesti. Sistem ima više tipova obavijesti: kreiranje rezervacije, izmjena rezervacije, produženje rezervacije, otkazivanje rezervacije i podsjetnik. Zbog toga je logično izdvojiti kreiranje obavijesti u posebnu klasu.

Na dijagram su dodani:

IObavijestFactory
EmailObavijestFactory

EmailObavijestFactory kreira odgovarajuću email obavijest na osnovu tipa obavijesti i podataka rezervacije.

Korištenje:

Factory Method je korišten zato što pojednostavljuje kreiranje različitih email obavijesti. Controller ili facade ne moraju znati kako se formira naslov i sadržaj svake poruke, nego samo pozovu factory klasu.

3. Abstract Factory

Abstract Factory patern koristi se kada treba kreirati porodice povezanih objekata bez direktnog navođenja konkretnih klasa. Pogodan je kada sistem ima više varijanti iste grupe objekata.

U sistemu pametnog parkinga ovaj patern bi se mogao koristiti ako bismo imali različite tipove parking sistema, npr. javni parking, privatni parking i garažni parking. Svaki od njih mogao bi imati svoje tipove parking mjesta, cjenovnike i pravila rezervacije.

Korištenje:

Abstract Factory trenutno nije odabran za naš dijagram jer sistem nema više porodica povezanih objekata koje bi zahtijevale posebne fabrike. Trenutno je dovoljan jednostavniji Factory Method za kreiranje email obavijesti.

4. Builder

Builder patern koristi se za kreiranje složenih objekata korak po korak. Posebno je koristan kada objekat ima više atributa ili kada se ne popunjavaju svi podaci odjednom.

U našem sistemu ovaj patern je odabran i dodat na klasni dijagram kod klase Rezervacija. Rezervacija sadrži više podataka: korisnika, parking mjesto, vrijeme početka i kraja, kontakt telefon, email, registracijske tablice, cijenu i status rezervacije. Zbog toga bi kreiranje rezervacije preko jednog velikog konstruktora bilo nepregledno.

Na dijagram je dodana klasa:

RezervacijaBuilder

Builder postepeno postavlja podatke rezervacije i na kraju metodom build() vraća gotov objekat Rezervacija.

Korištenje:

Builder je korišten zato što proces kreiranja rezervacije nije potpuno jednostavan. Potrebno je postaviti više podataka, provjeriti ih i izračunati cijenu. Builder omogućava da se rezervacija kreira pregledno i kontrolisano.

5. Prototype

Prototype patern koristi se kada se novi objekti kreiraju kopiranjem postojećeg objekta. Koristan je kada je kreiranje objekta skupo ili kada postoji mnogo sličnih objekata.

U sistemu pametnog parkinga Prototype bi se mogao koristiti za ponavljanje prethodne rezervacije. Na primjer, ako korisnik često rezerviše isto parking mjesto u slično vrijeme, sistem bi mogao kopirati postojeću rezervaciju i promijeniti samo datum ili vrijeme.

Korištenje:

Prototype nije dodat na dijagram jer u trenutnom sistemu kreiranje rezervacije nije zasnovano na kopiranju postojećih rezervacija, nego na unosu novih podataka. Za naš zadatak Builder je bolje rješenje jer se rezervacija kreira korak po korak.

Zaključak

U ovom zadatku su razmotreni osnovni kreacijski dizajn paterni i njihova moguća primjena u sistemu pametnog parkinga. Na novi klasni dijagram dodana su dva paterna: Factory Method i Builder.

Factory Method je odabran jer sistem ima više tipova email obavijesti i potrebno je njihovo kreiranje izdvojiti na jedno mjesto. Builder je odabran jer je rezervacija složen objekat sa više podataka i bolje ju je kreirati postepeno.

Ostali kreacijski paterni, kao što su Singleton, Abstract Factory i Prototype, također imaju moguće primjene u sistemu, ali nisu odabrani za dijagram jer nisu najvažniji za trenutno proširenje sistema.